JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface ChronoPeriod

All Superinterfaces:

TemporalAmount

All Known Implementing Classes:

Period

```
public interface ChronoPeriod
extends TemporalAmount
```

A date-based amount of time, such as '3 years, 4 months and 5 days' in an arbitrary chronology, intended for advanced globalization use cases.

This interface models a date-based amount of time in a calendar system. While most calendar systems use years, months and days, some do not. Therefore, this interface operates solely in terms of a set of supported units that are defined by the `Chronology`. The set of supported units is fixed for a given chronology. The amount of a supported unit may be set to zero.

The period is modeled as a directed amount of time, meaning that individual parts of the period may be negative.

Implementation Requirements:

This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.

Since:

1.8

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
Temporal	**addTo**(Temporal temporal)	Adds this period to the specified temporal object.
static ChronoPeriod	**between**(ChronoLocalDate startDa ChronoLocalDate endDateE	Obtains a ChronoPeriod consisting of amount of time between two dates.
boolean	**equals**(Object obj)	

		Checks if this period is equal to another period, including the chronology.
long	**get**(TemporalUnit unit)	Gets the value of the requested unit.
Chronology	**getChronology**()	Gets the chronology that defines the meaning of the supported units.
List<TemporalUnit>	**getUnits**()	Gets the set of units supported by this period.
int	**hashCode**()	A hash code for this period.
default boolean	**isNegative**()	Checks if any of the supported units of this period are negative.
default boolean	**isZero**()	Checks if all the supported units of this period are zero.
ChronoPeriod	**minus**(TemporalAmount amountTo	Returns a copy of this period with the specified period subtracted.
ChronoPeriod	**multipliedBy**(int scalar)	Returns a new instance with each amount in this period in this period multiplied by the specified scalar.
default ChronoPeriod	**negated**()	Returns a new instance with each amount in this period negated.
ChronoPeriod	**normalized**()	Returns a copy of this period with the amounts of each unit normalized.
ChronoPeriod	**plus**(TemporalAmount amountTo	Returns a copy of this period with the specified period added.

Temporal	**subtractFrom**(Temporal temporal)	Subtracts this period from the specified temporal object.
String	**toString**()	Outputs this period as a String.

Method Details

between

```
static ChronoPeriod between(ChronoLocalDate startDateInclusive,
                            ChronoLocalDate endDateExclusive)
```

Obtains a ChronoPeriod consisting of amount of time between two dates.

The start date is included, but the end date is not. The period is calculated using ChronoLocalDate.until(ChronoLocalDate). As such, the calculation is chronology specific.

The chronology of the first date is used. The chronology of the second date is ignored, with the date being converted to the target chronology system before the calculation starts.

The result of this method can be a negative period if the end is before the start. In most cases, the positive/negative sign will be the same in each of the supported fields.

Parameters:

startDateInclusive - the start date, inclusive, specifying the chronology of the calculation, not null

endDateExclusive - the end date, exclusive, in any chronology, not null

Returns:

the period between this date and the end date, not null

See Also:

ChronoLocalDate.until(ChronoLocalDate)

get

```
long get(TemporalUnit unit)
```

Gets the value of the requested unit.

The supported units are chronology specific. They will typically be YEARS, MONTHS and DAYS. Requesting an unsupported unit will throw an exception.

Specified by:

`get` in interface `TemporalAmount`

Parameters:

`unit` - the `TemporalUnit` for which to return the value

Returns:

the long value of the unit

Throws:

`DateTimeException` - if the unit is not supported

`UnsupportedTemporalTypeException` - if the unit is not supported

getUnits

`List<TemporalUnit> getUnits()`

Gets the set of units supported by this period.

The supported units are chronology specific. They will typically be `YEARS`, `MONTHS` and `DAYS`. They are returned in order from largest to smallest.

This set can be used in conjunction with `get(TemporalUnit)` to access the entire state of the period.

Specified by:

`getUnits` in interface `TemporalAmount`

Returns:

a list containing the supported units, not null

getChronology

`Chronology getChronology()`

Gets the chronology that defines the meaning of the supported units.

The period is defined by the chronology. It controls the supported units and restricts addition/subtraction to `ChronoLocalDate` instances of the same chronology.

Returns:

the chronology defining the period, not null

isZero

`default boolean isZero()`

Checks if all the supported units of this period are zero.

Returns:

true if this period is zero-length

isNegative

```
default boolean isNegative()
```

Checks if any of the supported units of this period are negative.

Returns:

true if any unit of this period is negative

plus

```
ChronoPeriod plus(TemporalAmount amountToAdd)
```

Returns a copy of this period with the specified period added.

If the specified amount is a `ChronoPeriod` then it must have the same chronology as this period. Implementations may choose to accept or reject other `TemporalAmount` implementations.

This instance is immutable and unaffected by this method call.

Parameters:

`amountToAdd` - the period to add, not null

Returns:

a `ChronoPeriod` based on this period with the requested period added, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

minus

```
ChronoPeriod minus(TemporalAmount amountToSubtract)
```

Returns a copy of this period with the specified period subtracted.

If the specified amount is a `ChronoPeriod` then it must have the same chronology as this period. Implementations may choose to accept or reject other `TemporalAmount` implementations.

This instance is immutable and unaffected by this method call.

Parameters:

`amountToSubtract` - the period to subtract, not null

Returns:

a `ChronoPeriod` based on this period with the requested period subtracted, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

multipliedBy

`ChronoPeriod multipliedBy(int scalar)`

Returns a new instance with each amount in this period in this period multiplied by the specified scalar.

This returns a period with each supported unit individually multiplied. For example, a period of "2 years, -3 months and 4 days" multiplied by 3 will return "6 years, -9 months and 12 days". No normalization is performed.

Parameters:

`scalar` - the scalar to multiply by, not null

Returns:

a `ChronoPeriod` based on this period with the amounts multiplied by the scalar, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

negated

`default ChronoPeriod negated()`

Returns a new instance with each amount in this period negated.

This returns a period with each supported unit individually negated. For example, a period of "2 years, -3 months and 4 days" will be negated to "-2 years, 3 months and -4 days". No normalization is performed.

Returns:

a `ChronoPeriod` based on this period with the amounts negated, not null

Throws:

`ArithmeticException` - if numeric overflow occurs, which only happens if one of the units has the value `Long.MIN_VALUE`

normalized

`ChronoPeriod normalized()`

Returns a copy of this period with the amounts of each unit normalized.

The process of normalization is specific to each calendar system. For example, in the ISO calendar system, the years and months are normalized but the days are
not, such that "15 months" would be normalized to "1 year and 3 months".

This instance is immutable and unaffected by this method call.

Returns:

a `ChronoPeriod` based on this period with the amounts of each unit normalized, not null

Throws:

`ArithmeticException` - if numeric overflow occurs

addTo

`Temporal addTo(Temporal temporal)`

Adds this period to the specified temporal object.

This returns a temporal object of the same observable type as the input with this period added.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.plus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisPeriod.addTo(dateTime);
dateTime = dateTime.plus(thisPeriod);
```

The specified temporal must have the same chronology as this period. This returns a temporal with the non-zero supported units added.

This instance is immutable and unaffected by this method call.

Specified by:

`addTo` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to add

`ArithmeticException` - if numeric overflow occurs

subtractFrom

`Temporal subtractFrom(Temporal temporal)`

Subtracts this period from the specified temporal object.

This returns a temporal object of the same observable type as the input with this period subtracted.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.minus(TemporalAmount)`.

```
// these two lines are equivalent, but the second approach is recommended
dateTime = thisPeriod.subtractFrom(dateTime);
dateTime = dateTime.minus(thisPeriod);
```

The specified temporal must have the same chronology as this period. This returns a temporal with the non-zero supported units subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`subtractFrom` in interface `TemporalAmount`

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to subtract

`ArithmeticException` - if numeric overflow occurs

equals

`boolean equals(Object obj)`

Checks if this period is equal to another period, including the chronology.

Compares this period with another ensuring that the type, each amount and the chronology are the same. Note that this means that a period of "15 Months" is not equal to a period of "1 Year and 3 Months".

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other period

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
int hashCode()
```

A hash code for this period.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

toString

```
String toString()
```

Outputs this period as a `String`.

The output will include the period amounts and chronology.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this period, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.